

Viruses and other malware often exploit bugs known as buffer overflows in widely used software. One method of preventing these exploits in software running on current Windows versions is Data Execution Prevention (DEP). Investigate buffer overflows and DEP. How does DEP prevent malicious attacks via buffer overflows? Is it always effective? Why might a user or administrator disable DEP?

In order to understand why Data Execution Prevention (DEP) protocols are useful against buffer overflow attacks, we must understand how these attacks override a system's code execution and how attackers use buffer overflows to execute malicious code. A buffer is a pre-allocated block of memory that is used to store a chunk of data while it's being processed or moved. Buffers are so fundamental to software that nearly every program relies on them. A buffer overflow occurs when a program writes more data onto a buffer than what the buffer size allows for. For example, a program writing an 8-byte block of data onto a 4-byte buffer would cause a buffer overflow. This causes the excess data to leak into the adjacent memory location, leading to software errors and crashes. Attackers use this error in software design to execute malicious code with the permission of the affected program. For example, if a program was running with admin (or root) privileges, it would be an ideal target of a buffer overflow attack because any code that is successfully executed through an attack on said program will run with those same privileges, giving the attacker immense power. A buffer overflow attack (BOA) works by overriding the return address and variable stack data in the targeted stack frame. To explain this, let's start by understanding what a stack frame is. When, for example, user input is processed by a function, the data is stored internally as a stack frame. A stack frame is a specific chunk of data that is made up of the buffer (for the actual user input), local variables that are used by the function, and the return address, which tells the function where to jump to after executing the function. An attacker utilizes functions that do not check for size before processing data, like the `gets()` function in C. Take the following function for example:

```
void login() {  
    char username[16]; //creates a buffer for the username with a size of 16 bytes.  
    gets(username); //calls the gets function on the username, gets() does not check for  
    size  
}
```

The way that a hacker will exploit this function is by passing a deliberately crafted payload (input) that:

- a. Fills in just enough space to fill in the buffer (16 bytes in this case).
- b. Overwrites the return address with a memory address of the attacker's choosing.
- c. Delivers the malicious shell code to be executed.

The data execution prevention security feature is a Windows feature that works to prevent BOAs by not allowing any code (shell or otherwise) to be executable when it is located in a memory region designated for data storage, most commonly in the stack and the heap. This mitigates the biggest advantage that BOAs provide: code execution with the privileges of trusted programs. This doesn't always stop the attack, however. Modern day BOA attacks use what is known as return oriented programming (ROP), a method of executing small, preexisting, and trusted code snippets usually ending in the RET (return) instruction. ROP pieces these instructions in order to execute malicious code. DEP is only truly resilient to these types of attacks when working in concert with Address Space Layout Randomization (ASLR), which randomizes where these instructions are stored. A deeper examination of ROP attacks is technically outside the scope of this short paper.

The biggest reason by far that an admin might disable DEP is legacy software compatibility. As dangerous as it is from a cyber defense perspective, there are a lot of machines running older operating systems such as Windows XP and Windows 7. Namely in fields like healthcare, where Windows XP is commonly seen in computers used in healthcare equipment. This is due to the equipment only being certified to run certain (often older) operating systems, and the cost of recertifying the equipment is too much for the hospitals. The main defense in this context is logical segmentation of the machines from the internet and hospital network or air gapping them altogether. Because DEP is a relatively newer feature in Windows operating systems, most of these systems have the feature disabled (or are unable to run it) for software compatibility. Similarly, industrial machinery also runs older operating systems, and are unable to support DEP.